



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش کار آزمایش هشتم

عنوان:

طراحی واحد محاسبه و منطق مختلط

نام درس:

آزمایشگاه طراحی سامانه‌های رقمی

تابستان ۱۴۰۵

استاد:

دکتر اجلالی

دستیار استاد:

علی اثنی عشری

گروه ۲

علی سلطانی - ۴۰۳۱۰۶۰۹۲

یگانه کربلایی - ۴۰۳۱۰۶۵۲۷

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	هدف و صورت آزمایش	۳
۱.۱	چالش ساختاری و راهبرد طراحی	۳
۲	پیمانه جمع و تفریق اعداد علامت دار	۳
۱.۲	پیاده سازی کد	۳
۲.۲	تحلیل منطق تشخیص سرریز	۴
۳.۲	آزمون مقیاس پذیری و شبیه سازی	۴
۱.۳.۲	مورد آزمون نخست: محاسبات اعداد کوچک (بدون سرریز)	۴
۲.۳.۲	مورد آزمون دوم: بررسی سرریز در سیستم ۴ بیتی	۴
۳.۳.۲	مورد آزمون سوم: بررسی سرریز در سیستم ۸ بیتی	۴
۴.۳.۲	مورد آزمون چهارم: بررسی سرریز در سیستم ۱۶ بیتی	۵
۵.۳.۲	مورد آزمون پنجم: سرریز منفی در عمل تفریق	۵
۳	پیمانه ضرب کننده	۵
۱.۳	پیاده سازی کد	۵
۲.۳	تحلیل پدیده برش و عدم سرریز	۶
۳.۳	آزمون مقیاس پذیری و شبیه سازی	۶
۱.۳.۳	دسته ی نخست: مقادیر مثبت کوچک	۶
۲.۳.۳	دسته ی دوم: بیشترین مقادیر مرزی مثبت	۶
۳.۳.۳	دسته ی سوم: بیشترین مقادیر مرزی منفی	۶
۴	پیمانه مزدوج مختلط	۶
۱.۴	پیاده سازی کد	۷
۲.۴	تحلیل منطق سرریز	۷
۳.۴	آزمون مقیاس پذیری و شبیه سازی	۷
۱.۳.۴	مورد آزمون نخست: مزدوج عادی	۷
۲.۳.۴	مورد آزمون دوم: سرریز در نمونه ۴ بیتی	۷
۳.۳.۴	مورد آزمون سوم و چهارم: سرریز در نمونه ۸ و ۱۶ بیتی	۸
۵	پیمانه محاسبه اندازه مربع عدد مختلط	۸
۱.۵	پیاده سازی کد	۸
۲.۵	تحلیل منطق سرریز	۸
۳.۵	آزمون مقیاس پذیری و شبیه سازی	۸
۱.۳.۵	مورد آزمون نخست: مقادیر عادی	۹
۲.۳.۵	مورد آزمون دوم: سرریز حاصل جمع در ۴ بیت	۹
۳.۳.۵	مورد آزمون سوم: سرریز حاصل جمع در ۸ بیت	۹
۶	پیمانه مسیر داده	۹
۱.۶	پیاده سازی کد	۹
۲.۶	تحلیل ساختار مسیر داده	۱۲
۷	فرمت دستورها و کدهای عملیاتی	۱۲
۸	مدیریت پرچم های وضعیت	۱۲
۹	پیمانه واحد کنترل	۱۳
۱.۹	پیاده سازی کد	۱۳
۲.۹	زمان بندی و مراحل اجرای دستورات چندچرخه ای	۱۵
۱.۲.۹	دستور جمع دو عدد مختلط	۱۵
۲.۲.۹	دستور تفریق دو عدد مختلط	۱۵

۱۵	دستور محاسبه مزدوج مختلط یک عدد	۳.۲.۹
۱۶	دستور ضرب دو عدد مختلط	۴.۲.۹
۱۶	دستور محاسبه اندازه مربع عدد مختلط	۵.۲.۹
۱۶	آزمون نهایی سیستم	۱۰
۱۶	پیاده‌سازی کد میزآزمون نهایی	۱.۱۰
۱۷	تحلیل موارد آزمون (محتوای حافظه)	۲.۱۰
۱۸	دستور اول: جمع دو عدد مختلط مثبت	۱.۲.۱۰
۱۸	دستور دوم: جمع دو عدد مختلط منفی	۲.۲.۱۰
۱۸	دستور سوم: تفریق دو عدد مختلط با نتیجه منفی	۳.۲.۱۰
۱۸	دستور چهارم: ضرب دو عدد مختلط با مؤلفه‌های مثبت و منفی	۴.۲.۱۰
۱۹	دستور پنجم: محاسبه مزدوج مختلط	۵.۲.۱۰
۱۹	دستور ششم: محاسبه اندازه مربع عدد مختلط	۶.۲.۱۰
۱۹	دستور هفتم: بررسی خروجی صفر و فعال شدن پرچم مربوطه	۷.۲.۱۰
۱۹	دستور هشتم: بررسی حالت‌های مرزی برای وقوع سرریز	۸.۲.۱۰
۱۹	دستور نهم: بدون عملیات	۹.۲.۱۰
۲۰	آزمون مقیاس‌پذیری (۴ و ۱۶ بیتی)	۱۰.۲.۱۰
۲۱	بررسی قابلیت سنتز در نرم‌افزار Quartus	۱۱

۱ هدف و صورت آزمایش

هدف از این آزمایش، طراحی، توصیف سخت‌افزاری و پیاده‌سازی یک واحد منطق حسابی برای اعداد مختلط است. طراحی باید به صورت پیمانه‌ای^۱ انجام شود؛ به این معنا که هر بخش محاسباتی به صورت یک پیمانه مستقل طراحی شده و سپس با اتصال این پیمانه‌ها به یکدیگر یک ساختار کامل‌تر ساخته شود. در نهایت این واحد باید بتواند دستورهای مختلف محاسباتی مربوط به اعداد مختلط را از یک حافظه ۳۲ کلمه‌ای دریافت کرده و آن‌ها را اجرا کند.

۱.۱ چالش ساختاری و راهبرد طراحی

یکی از چالش‌های اصلی و ظریف این آزمایش، وجود یک دوگانگی ظاهری در دستورکار بود: از یک سو خواسته شده بود که برای هر عملیات یک پیمانه مستقل طراحی شود، و از سوی دیگر محدودیت وجود داشت که در تمام مسیر داده، نباید از واحد جمع‌کننده یا ضرب‌کننده بیش از یک بار استفاده شود.

برای حل این چالش و جلوگیری از تولید سخت‌افزار اضافی توسط نرم‌افزار سنتز، چهار ماژول پایه به صورت کاملاً مستقل پیاده‌سازی شده‌اند. با این حال، ماژول‌های مزدوج مختلط^۲ و اندازه مربع^۳ فاقد عملگرهای سخت‌افزاری بوده و صرفاً به عنوان واحدهای پردازش مسیر داده عمل می‌کنند که خروجی‌های خود را از ماژول‌های مرکزی (یک ضرب‌کننده و یک جمع‌کننده) تغذیه کرده و منطق سرریز^۴ مخصوص به خود را اعمال می‌کنند.

۲ پیمانه جمع و تفریق اعداد علامت‌دار

این پیمانه وظیفه انجام عملیات پایه جمع و تفریق را در سیستم بر عهده دارد. با استفاده از سیگنال کنترلی، مدار تصمیم می‌گیرد که آیا عملوندها باید جمع شوند یا تفریق. بخش حقیقی و بخش موهومی هر عدد مختلط به صورت عدد علامت‌دار در نمایش مکمل دوه^۵ در نظر گرفته شده است.

۱.۲ پیاده‌سازی کد

```
1 module AddSub #(parameter WIDTH = 8) (  
2     input signed [WIDTH-1:0] a,  
3     input signed [WIDTH-1:0] b,  
4     input op, // 0: Add, 1: Sub  
5     output signed [WIDTH-1:0] result,  
6     output overflow  
7 );  
8     assign result = (op == 1'b0) ? (a + b) : (a - b);  
9  
10    wire sign_a = a[WIDTH-1];  
11    wire sign_b = b[WIDTH-1];  
12    wire sign_r = result[WIDTH-1];  
13  
14    assign overflow = (op == 1'b0) ?  
15        (~sign_a & ~sign_b & sign_r) | (sign_a & sign_b & ~sign_r) :  
16        (~sign_a & sign_b & sign_r) | (sign_a & ~sign_b & ~sign_r);  
17 endmodule
```

کد پیمانه جمع و تفریق

توضیح خط به خط کد:

^۱Modular

^۲Complex Conjugate

^۳Magnitude Squared

^۴Overflow

^۵Two's Complement

- خطوط ۱ تا ۷: در این بخش، درگاه‌های ورودی شامل عملوندها و عملگر کنترلی (op)، و همچنین درگاه‌های خروجی برای نتیجه و پرچم سرریز تعریف شده‌اند. کلمه کلیدی signed برای اعمال رفتار محاسبات مکمل دو ضروری است.
- خط ۸: این خط به وسیله یک عبارت شرطی ترکیبی مشخص می‌کند که اگر $op=0$ باشد مدار به عنوان جمع‌کننده و اگر $op=1$ باشد به عنوان تفریق‌کننده عمل کند.
- خطوط ۱۰ تا ۱۲: استخراج پرارزش‌ترین بیت (بیت علامت) برای عملوندها و نتیجه جهت بررسی وضعیت سرریز انجام شده است.
- خطوط ۱۴ تا ۱۶: پیاده‌سازی منطق ترکیبی جبر بول برای تشخیص سرریز. در حالت جمع، اگر دو عدد هم علامت باشند اما نتیجه علامت متفاوتی داشته باشد سرریز رخ داده است. در حالت تفریق نیز مدار به صورت سخت‌افزاری تغییر علامت عملوند دوم را لحاظ می‌کند.

۲.۲ تحلیل منطق تشخیص سرریز

سرریز زمانی رخ می‌دهد که حاصل جمع دو عدد هم علامت، علامتی متفاوت با آن‌ها داشته باشد. جدول زیر به طور دقیق نشان می‌دهد در چه ترکیب‌هایی از علامت‌ها، سرریز خواهد داد:

عملیات (op)	علامت A	علامت B	علامت نتیجه (بدون در نظر گرفتن سرریز)	وضعیت سرریز
جمع (۰)	مثبت (۰)	مثبت (۰)	منفی (۱)	رخ می‌دهد (۱)
جمع (۰)	منفی (۱)	منفی (۱)	مثبت (۰)	رخ می‌دهد (۱)
تفریق (۱)	مثبت (۰)	منفی (۱)	منفی (۱)	رخ می‌دهد (۱)
تفریق (۱)	منفی (۱)	مثبت (۰)	مثبت (۰)	رخ می‌دهد (۱)
سایر حالات	-	-	-	رخ نمی‌دهد (۰)

جدول ۱: جدول درستی وضعیت‌های منجر به سرریز در جمع و تفریق

۳.۲ آزمون مقیاس‌پذیری و شبیه‌سازی

برای اثبات مقیاس‌پذیری، فایل میزآزمون tb_AddSub.v دارای سه نمونه‌سازی با عرض‌های ۴، ۸ و ۱۶ بیت است. در ادامه ۵ مورد آزمون پیاده‌سازی شده در کد تحلیل می‌گردند:

۱.۳.۲ مورد آزمون نخست: محاسبات اعداد کوچک (بدون سرریز)

ورودی‌ها: $a = 3$ و $b = 4$ با عملگر جمع ($op = 0$). در این حالت خروجی مورد انتظار برابر با ۷ است. از آنجا که عدد ۷ در محدوده هر سه سیستم (۴، ۸ و ۱۶ بیتی) می‌گنجد، پرچم سرریز در تمامی نسخه‌ها صفر باقی می‌ماند.

۲.۳.۲ مورد آزمون دوم: بررسی سرریز در سیستم ۴ بیتی

ورودی‌ها: $a = 7$ و $b = 2$ با عملگر جمع ($op = 0$). حاصل جمع برابر با ۹ است. از آنجا که بیشترین مقدار مثبت قابل نمایش در یک سیستم ۴ بیتی علامت‌دار برابر با ۷ است، انتظار می‌رود که خروجی نمونه ۴ بیتی دچار سرریز شده و پرچم آن یک می‌گردد اما در ۸ و ۱۶ بیت مشکلی نیست.

۳.۳.۲ مورد آزمون سوم: بررسی سرریز در سیستم ۸ بیتی

ورودی‌ها: $a = 100$ و $b = 50$ با عملگر جمع ($op = 0$). حاصل ۱۵۰ است. ماکزیمم مقدار ۸ بیتی ۱۲۷ است، پس پرچم سرریز ۸ بیتی روشن می‌شود. سیستم ۱۶ بیتی مقدار را به درستی محاسبه می‌کند. نکته در این تست این است که ماژول ۴ بیتی پرچم ovf4 را یک نمی‌کند؛ زیرا به دلیل نحوه تخصیص ورودی‌ها در کد میزآزمون (`a_val[3:0]`)، تنها ۴ بیت کم‌ارزش اعداد

به آن داده می‌شود (که برای ۱۰۰ معادل ۴ و برای ۵۰ معادل ۲ است). ماژول تنها جمع $6 = 4 + 2$ را می‌بیند که در ۴ بیت جا می‌شود. این یعنی بررسی سرریز تنها زمانی معتبر است که عرض گذرگاه^۶ بتواند خود عملوندها را در بر بگیرد.

۴.۳.۲ مورد آزمون چهارم: بررسی سرریز در سیستم ۱۶ بیتی

ورودی‌ها: $a = 20000$ و $b = 15000$ با عملگر جمع ($op = 0$). حاصل ۳۵۰۰۰ از حد ۱۶ بیتی (۳۲۷۶۷) فراتر می‌رود، در نتیجه پرچم سرریز ۱۶ بیتی نیز با موفقیت یک می‌شود.

۵.۳.۲ مورد آزمون پنجم: سرریز منفی در عمل تفریق

ورودی‌ها: $a = -100$ و $b = 50$ با عملگر تفریق ($op = 1$). حاصل تفریق -150 می‌شود. این عدد از مرز پایین ۸ بیتی (-128) عبور کرده و سرریز در آن رخ می‌دهد.

Time	Data	Operation
3	100	20000
4	50	15000
7	-7	-8
7	9	-106
7	150	-30536

شکل ۱: شکل موج خروجی میزآزمون ماژول جمع و تفریق برای عرض‌های مختلف

۳ پیمانه ضرب‌کننده

این پیمانه عملگر ضرب سیستم است. از آنجا که در عملیات ضرب، عرض بیت خروجی نسبت به عرض بیت ورودی افزایش می‌یابد، خروجی این ماژول دقیقاً دو برابر عرض ورودی تعریف شده است.

۱.۳ پیاده‌سازی کد

```

1 module Mul #(parameter WIDTH = 8) (
2     input signed [WIDTH-1:0] a,
3     input signed [WIDTH-1:0] b,
4     output signed [(2*WIDTH)-1:0] result
5 );
6     assign result = a * b;
7 endmodule

```

کد پیمانه ضرب‌کننده

توضیح خط به خط کد:

- خطوط ۱ تا ۵: تعریف عملوندهای ورودی با عرض WIDTH و تعریف درگاه خروجی با عرض $2 \times WIDTH$. این افزایش عرض گذرگاه، راهبرد اصلی برای جلوگیری از اعوجاج در ضرب است.

- خط ۶: تخصیص پیوسته^۷ با استفاده از عملگر $*$. این خط در زمان سنتز به یک بلوک ضرب‌کننده سخت‌افزاری ترکیبی تبدیل می‌شود که تمامی ارزش‌های مکانی تولید شده را در خروجی بسط‌یافته قرار می‌دهد.

^۶Bus

^۷Continuous Assignment

۲.۳ تحلیل پدیده برش و عدم سرریز

خروجی با عرض کامل نگهداری شده است و از روش قطع کردن^۸ بیتی برای خروجی استفاده نشده است، در نتیجه ضرب‌کننده نیازی به پرچم سرریز ندارد. با این حال، در میزآزمون، زمانی که عددی بزرگتر از ظرفیت ورودی ماژول به سیستم تغذیه می‌شود (مثلاً دادن عدد ۱۲۷ به ماژول ۴ بیتی)، به دلیل ساختار سیم‌کشی میزآزمون که صراحتاً نوشته شده $a(a_val[3:0])$ ، تنها ۴ بیت کم‌ارزش آن عدد به ماژول داده می‌شود. عدد ۱۲۷ در قالب ۴ بیت پایین خود به صورت 1111 (معادل ۱-) دیده می‌شود. بنابراین مدار عملاً در حال ضرب 1×-1 بوده و پاسخ ۱ را تولید می‌کند. این پدیده خطای ضرب‌کننده نیست، بلکه رفتار طبیعی برش بیت‌ها در ورودی پورت‌ها پیش از ورود به محاسبات است.

۳.۳ آزمون مقیاس‌پذیری و شبیه‌سازی

میزآزمون شامل سه نمونه ۴، ۸ و ۱۶ بیتی است و سه دسته کلی از مقادیر را مطابق کدهای موجود در فایل تست بررسی می‌کند:

۱.۳.۳ دسته‌ی نخست: مقادیر مثبت کوچک

تست با مقادیر $a = 3$ و $b = 2$ انجام شده که پاسخ ۶ در تمامی عرض‌ها با موفقیت ثبت می‌شود.

۲.۳.۳ دسته‌ی دوم: بیشترین مقادیر مرزی مثبت

ضرب بزرگترین عدد مثبت هر سیستم در خودش:

• ۴ بیتی: ضرب $7 \times 7 = 49$.

• ۸ بیتی: ضرب $127 \times 127 = 16129$.

• ۱۶ بیتی: ضرب $32767 \times 32767 = 1073676289$.

۳.۳.۳ دسته‌ی سوم: بیشترین مقادیر مرزی منفی

ضرب منفی‌ترین اعداد در خودشان که منجر به اعداد مثبت بسیار بزرگ می‌شود:

• ۴ بیتی: ضرب $-8 \times -8 = 64$.

• ۸ بیتی: ضرب $-128 \times -128 = 16384$.

• ۱۶ بیتی: ضرب $-32768 \times -32768 = 1073741824$.

در تمام این حالات خروجی دقیقاً در ظرفیت دوبرابر شده به درستی محاسبه می‌شود.

Mags									
/b_MU/a_val	No Data	3	7	127	32767	-8	-128	-32768	
/b_MU/b_val	No Data	2	7	127	32767	-8	-128	-32768	
/b_MU/res4	No Data	6	49	1		64	0		
/b_MU/res8	No Data	6	49	16129	1	64	16384	0	
/b_MU/res16	No Data	6	49	16129	1073676289	64	16384	1073741824	

شکل ۲: شکل موج خروجی میزآزمون ماژول ضرب‌کننده

۴ پیمانه مزدوج مختلط

به جای ساخت یک مدار متمم دو مستقل برای قرینه کردن بخش موهومی، این پیمانه نتیجه ضرب بخش موهومی در منفی یک را مستقیماً از ضرب‌کننده مرکزی دریافت کرده و به خروجی هدایت می‌کند تا شرط محدودیت منابع رعایت شود.

^۸Truncation

۱.۴ پیاده‌سازی کد

```
1 module Conj #(parameter WIDTH = 8) (  
2     input signed [WIDTH-1:0] A_real,  
3     input signed [WIDTH-1:0] A_imag,  
4     input signed [2*WIDTH-1:0] shared_mul_neg_imag,  
5     output signed [2*WIDTH-1:0] result_real,  
6     output signed [2*WIDTH-1:0] result_imag,  
7     output overflow_imag  
8 );  
9     assign result_real = {{WIDTH{A_real[WIDTH-1]}}, A_real};  
10    assign result_imag = shared_mul_neg_imag;  
11  
12    // Overflow occurs only when A_imag is the maximum negative value  
13    assign overflow_imag = (A_imag == {1'b1, {(WIDTH-1){1'b0}}});  
14 endmodule
```

کد پیمانانه مزدوج مختلط

توضیح خط به خط کد:

- خط ۹: بخش حقیقی برای تطابق با سائیز مسیر داده، به وسیله عملگرهای تکرار $\{\}$ بسط علامت^۹ می‌شود تا ارزش مکانی عدد در فضای دو برابر شده حفظ گردد.
- خط ۱۰: قرینه بخش موهومی که پیش‌تر توسط ضرب‌کننده محاسبه شده است، تنها از طریق یک سیم‌کشی مستقیم ترکیبی به درگاه خروجی اختصاص می‌یابد.
- خط ۱۳: منطق ترکیبی سرریز. در سیستم مکمل دو تنها عددی که قرینه‌اش در همان ظرفیت جا نمی‌شود، منفی‌ترین عدد است (مانند ۱۲۸-). این خط بررسی می‌کند که آیا بخش موهومی دقیقاً مساوی با الگوی دودویی^{۱۰} منفی‌ترین عدد (یک ۱ در پرازش‌ترین بیت و بقیه صفر) است یا خیر.

۲.۴ تحلیل منطق سرریز

در سیستم اعداد علامت‌دار، محدوده اعداد نامتقارن است. در صورتی که بخش موهومی عدد ورودی، منفی‌ترین مقدار ممکن در ظرفیت تعیین‌شده باشد، قرینه آن در همان ظرفیت قابل نمایش نخواهد بود. از آنجا که ماژول فرض می‌کند عدد نهایی باید ماهیت WIDTH بیتی داشته باشد، با دریافت این مقدار مرزی، سیگنال سرریز را فعال می‌سازد.

۳.۴ آزمون مقیاس‌پذیری و شبیه‌سازی

میزآزمون شامل سه نمونه با عرض‌های ۴، ۸ و ۱۶ بیت است تا عملکرد مدار در قبال حالت‌های مرزی اثبات شود. سیگنال محاسبه قرینه به صورت دستی در میزآزمون تزریق شده است.

۱.۳.۴ مورد آزمون نخست: مزدوج عادی

ورودی‌ها: حقیقی ۳ و موهومی ۲. مدار قرینه را به صورت $z - 3$ در تمامی نمونه‌ها ثبت کرده و سرریز صفر می‌ماند.

۲.۳.۴ مورد آزمون دوم: سرریز در نمونه ۴ بیتی

بخش موهومی ورودی ۸- داده می‌شود. چون قرینه آن (۸+) در ۴ بیت جای نمی‌گیرد، ماژول ۴ بیتی پرچم سرریز خود را یک می‌کند.

^۹Sign Extension

^{۱۰}Binary

۳.۳.۴ مورد آزمون سوم و چهارم: سرریز در نمونه ۸ و ۱۶ بیتی

با دادن مقادیر ۱۲۸- به نمونه ۸ بیتی و ۳۲۷۶۸- به نمونه ۱۶ بیتی، هر یک از این ماژول‌ها تشخیص می‌دهند که عملوند خارج از حد تقارن است و پرچم خود را فعال می‌کنند.

[illegible]

شکل ۳: شکل موج خروجی میزآزمون ماژول مزدوج مختلط

۵ پیمانہ محاسبہ اندازہ مربع عدد مختلط

این پیمانه مقادیر توان دوم و حاصل جمع آن‌ها را با اشتراک‌گذاری منابع سخت‌افزاری از ماژول‌های مرکزی دریافت می‌کند.

۱.۵ پیاده‌سازی کد

```

1 module Mag2 #(parameter WIDTH = 8) (
2     input signed [2*WIDTH-1:0] a2,
3     input signed [2*WIDTH-1:0] b2,
4     input signed [2*WIDTH-1:0] shared_add_sum,
5     output signed [2*WIDTH-1:0] result,
6     output overflow
7 );
8     assign result = shared_add_sum;
9     // Overflow check: sum of squares is always positive, so MSB should be 0
10    assign overflow = shared_add_sum[2*WIDTH-1];
11 endmodule

```

کد پیمانه مجذور اندازه

توضیح خط به خط کد:

- **خط ۸:** نتیجه نهایی مستقیماً از خروجی مازول جمع‌کننده که در مسیر داده قرار دارد کپی می‌شود و هیچ عملیات سخت‌افزاری مضاعفی تولید نمی‌گردد.
- **خط ۱۰:** از آنجا که جمع دو مقدار مربع (توان دوم) قطعاً مقداری مثبت است، هرگاه پرارزش‌ترین بیت نتیجه ۱ شود به معنای تغییر علامت ناخواسته ناشی از کمبود ظرفیت (سرریز) است. لذا این خط مستقیماً بیت پرارزش را به درگاه overflow اختصاص می‌دهد.

۲.۵ تحلیل منطق سرریز

همان‌طور که در کد اشاره شد، تنها شرط سرریز در این ماژول، منفی شدن حاصل‌جمع (به دلیل خروج از ظرفیت مدار) است که از روی ارزش علامت در `shared_add_sum` بررسی می‌شود. البته در تست‌بنچ برای اعداد بسیار بزرگ در ماژول ۴ بیتی (مثل ۲۰۰۰۰)، به علت وراثت و برش بیت‌ها در ورودی پورت، چهار بیت پایین آن مقدار برابر صفر شده و مدار جمع $0 + 0 = 0$ را می‌پسند که سرریز ندارد؛ اما برای ۸ و ۱۶ بیت سرریز روشن می‌شود.

۳.۵ آزمون مقیاس‌پذیری و شبیه‌سازی

آزمون‌های مقیاس‌پذیری به شرح زیر بررسی شدند:

ورودی‌های مربع ۴ و ۹ که مجموع آن‌ها ۱۳ است. مدار مقدار ۱۳ را بدون خط خروجی می‌دهد.

مقادیر مربع ۱۰۰ که مجموع آن‌ها ۲۰۰ می‌شود. در نمونه ۴ بیتی که خروجی ۸ بیتی دارد (حداکثر ۱۲۷)، مقدار ۲۰۰ باعث منفی شدن علامت باینری شده و سرریز در مدار ۴ بیتی فعال می‌گردد.

مقادیر مربع ۲۰۰۰۰ که مجموع آنها ۴۰۰۰۰ است. خروجی نهایی سیستم ۸ بیتی دارای ۱۶ بیت عرض است (حداکثر ظرفیت ۳۲۷۶۷). مقدار ۴۰۰۰۰ باعث سرریز شده و پرچم با موفقیت فعال می‌گردد.

[illegible]

شکل ۴: شکل موج خروجی میزآزمون ماژول مجذور اندازه

پیمانه مسیر داده، محل استقرار فیزیکی عملگراست. این پیمانه با فرمان‌پذیری از واحد کنترل، ورودی‌های مناسب را به سمت تنها ضرب‌کننده و جمع‌کننده موجود در مدار هدایت می‌کند.

```

1 module Datapath #(parameter WIDTH = 8) (
2     input clk,
3     input rst,
4     input [2:0] opcode,
5     input signed [WIDTH-1:0] A_real,
6     input signed [WIDTH-1:0] A_imag,
7     input signed [WIDTH-1:0] B_real,
8     input signed [WIDTH-1:0] B_imag,
9
10    input [2:0] sel_mul_1,
11    input [2:0] sel_mul_2,
12    input [2:0] sel_add_1,
13    input [2:0] sel_add_2,
14
15    input add_sub_ctrl,
16
17    input en_reg1, en_reg2, en_reg3, en_reg4,
18    input en_addout_real, en_addout_imag, en_mulout_real, en_mulout_imag,
19
20    output reg signed [2*WIDTH-1:0] out_real,
21    output reg signed [2*WIDTH-1:0] out_imag,
22    output zero,
23    output overflow,
24    output negative_real,
25    output negative_imag
26 );

```

```

37 wire signed [2*WIDTH-1:0] A_real_ext = {{WIDTH{A_real[WIDTH-1]}}, A_real};
38 wire signed [2*WIDTH-1:0] A_imag_ext = {{WIDTH{A_imag[WIDTH-1]}}, A_imag};
39 wire signed [2*WIDTH-1:0] B_real_ext = {{WIDTH{B_real[WIDTH-1]}}, B_real};
40 wire signed [2*WIDTH-1:0] B_imag_ext = {{WIDTH{B_imag[WIDTH-1]}}, B_imag};
41
42 reg signed [2*WIDTH-1:0] reg1, reg2, reg3, reg4;
43
44 reg signed [WIDTH-1:0] mul_in1, mul_in2;
45 always @(*) begin
46     case(sel_mul_1)
47         3'b000: mul_in1 = A_real;
48         3'b001: mul_in1 = A_imag;
49         3'b010: mul_in1 = B_real;
50         3'b011: mul_in1 = B_imag;
51         3'b100: mul_in1 = {WIDTH{1'b1}}; // -1
52         default: mul_in1 = 0;
53     endcase
54
55     case(sel_mul_2)
56         3'b000: mul_in2 = A_real;
57         3'b001: mul_in2 = A_imag;
58         3'b010: mul_in2 = B_real;
59         3'b011: mul_in2 = B_imag;
60         3'b100: mul_in2 = {WIDTH{1'b1}}; // -1
61         default: mul_in2 = 0;
62     endcase
63 end
64
65 wire signed [2*WIDTH-1:0] mul_out;
66 Mul #(.WIDTH(WIDTH)) mult_unit (.a(mul_in1), .b(mul_in2), .result(mul_out));
67
68 reg signed [2*WIDTH-1:0] add_in1, add_in2;
69 always @(*) begin
70     case(sel_add_1)
71         3'b000: add_in1 = A_real_ext;
72         3'b001: add_in1 = A_imag_ext;
73         3'b010: add_in1 = B_real_ext;
74         3'b011: add_in1 = B_imag_ext;
75         3'b100: add_in1 = reg1;
76         3'b101: add_in1 = reg2;
77         3'b110: add_in1 = reg3;
78         3'b111: add_in1 = reg4;
79         default: add_in1 = 0;
80     endcase
81
82     case(sel_add_2)
83         3'b000: add_in2 = A_real_ext;
84         3'b001: add_in2 = A_imag_ext;
85         3'b010: add_in2 = B_real_ext;
86         3'b011: add_in2 = B_imag_ext;
87         3'b100: add_in2 = reg1;
88         3'b101: add_in2 = reg2;
89         3'b110: add_in2 = reg3;
90         3'b111: add_in2 = reg4;
91         default: add_in2 = 0;
92     endcase
93 end
94
95 wire signed [2*WIDTH-1:0] add_out;
96 wire add_overflow;
97 AddSub #(.WIDTH(2*WIDTH)) addsub_unit (.a(add_in1), .b(add_in2), .op(
98     add_sub_ctrl), .result(add_out), .overflow(add_overflow));

```

```

۸۹
۹۰ wire signed [2*WIDTH-1:0] conj_real, conj_imag;
۹۱ wire conj_overflow;
۹۲ Conj #(.WIDTH(WIDTH)) conj_unit (.A_real(A_real), .A_imag(A_imag), .
    shared_mul_neg_imag(mul_out),
۹۳                                     .result_real(conj_real), .result_imag(conj_imag
    ), .overflow_imag(conj_overflow));
۹۴
۹۵ wire signed [2*WIDTH-1:0] mag2_result;
۹۶ wire mag2_overflow;
۹۷ Mag2 #(.WIDTH(WIDTH)) mag2_unit (.a2(reg1), .b2(reg2), .shared_add_sum(add_out),
۹۸                                     .result(mag2_result), .overflow(mag2_overflow))
    ;
۹۹
۱۰۰ always @(posedge clk or posedge rst) begin
۱۰۱     if (rst) begin
۱۰۲         reg1 <= 0; reg2 <= 0; reg3 <= 0; reg4 <= 0;
۱۰۳         out_real <= 0; out_imag <= 0;
۱۰۴     end else begin
۱۰۵         if (en_reg1) reg1 <= mul_out;
۱۰۶         if (en_reg2) reg2 <= mul_out;
۱۰۷         if (en_reg3) reg3 <= mul_out;
۱۰۸         if (en_reg4) reg4 <= mul_out;
۱۰۹
۱۱۰         if (en_addout_real) begin
۱۱۱             if (opcode == 3'b011) out_real <= conj_real;
۱۱۲             else if (opcode == 3'b100) out_real <= mag2_result;
۱۱۳             else out_real <= add_out;
۱۱۴         end
۱۱۵         else if (en_mulout_real) out_real <= mul_out;
۱۱۶
۱۱۷         if (en_addout_imag) out_imag <= add_out;
۱۱۸         else if (en_mulout_imag) begin
۱۱۹             if (opcode == 3'b011) out_imag <= conj_imag;
۱۲۰             else out_imag <= mul_out;
۱۲۱         end
۱۲۲     end
۱۲۳ end
۱۲۴
۱۲۵ assign zero = (out_real == 0 && out_imag == 0) ? 1'b1 : 1'b0;
۱۲۶ assign negative_real = out_real[2*WIDTH-1];
۱۲۷ assign negative_imag = out_imag[2*WIDTH-1];
۱۲۸ assign overflow = (opcode == 3'b011) ? conj_overflow :
۱۲۹                     (opcode == 3'b100) ? mag2_overflow : add_overflow;
۱۳۰
۱۳۱ endmodule

```

کد پیمانه مسیر داده

توضیح خط به خط کد:

- خطوط ۳۴ تا ۳۸: تعریف ثبات‌های چهارگانه برای ذخیره نتایج میانی در عملیات‌های چندچرخه‌ای (مانند ضرب). این ثبات‌ها به عنوان حافظه داخلی مسیر داده عمل می‌کنند.
- خطوط ۳۷ تا ۶۰: پیاده‌سازی تسهیم‌کننده‌های^{۱۱} ورودی ضرب‌کننده با استفاده از بلوک ترکیبی (*). always. این مقادیر بر اساس سیگنال‌های انتخابی واحد کنترل هدایت می‌شوند.
- خطوط ۶۳ و ۹۰: نمونه‌سازی متمرکز و یکتای ضرب‌کننده و جمع‌کننده. این پیاده‌سازی تضمین می‌کند که از نظر سخت‌افزاری تنها یک نمونه از هر کدام سنتز شود.

^{۱۱}Multiplexer

- خطوط ۱۰۰ تا ۱۳۰: بلوک منطق ترتیبی (با تحریک لبه بالارونده ساعت). این بلوک وظیفه ایجاد فلیپ‌فلاپ‌های مورد نیاز برای نگهداری مقادیر ثبات‌ها و خروجی‌های نهایی مدار را بر عهده دارد.
- خطوط ۱۳۳ تا ۱۳۸: منطق ترکیبی برای تولید پرچم‌های وضعیت. در اینجا یک تسهیم‌کننده هوشمند بر اساس نوع دستور، سرریز صحیح را به خروجی سیستم هدایت می‌کند.

۲.۶ تحلیل ساختار مسیر داده

در این پیمانه دقیقاً یک بار از پیمانه ضرب و یک بار از پیمانه جمع استفاده شده است که تضمین‌کننده رعایت قید طراحی است. همچنین پیمانه‌های مجازی مزدوج و مجذور اندازه با تغذیه از خروجی‌های این دو عملگر، پردازش خود را انجام می‌دهند. بلوک‌های منطق ترتیبی نیز با گرفتن سیگنال فعال‌ساز، نتایج میانی را برای چرخه بعدی در ثبات‌های چهارگانه حفظ می‌کنند.

۷ فرمت دستورها و کدهای عملیاتی

برای ساده‌سازی طراحی ماشین فرضی، یک ساختار دقیق برای قالب دستور در نظر گرفته شده است. طول هر دستور ۳۵ بیت است تا بتواند عملوندها را به صورت مستقیم در خود جای دهد. ساختار ۳۵ بیتی هر دستور در حافظه به شرح زیر است:

- بیت‌های ۳۴ تا ۳۲ (۳ بیت): کد دستور^{۱۲} که نوع عملیات را مشخص می‌کند.
- بیت‌های ۳۱ تا ۲۴ (۸ بیت): بخش حقیقی عملوند اول.
- بیت‌های ۲۳ تا ۱۶ (۸ بیت): بخش موهومی عملوند اول.
- بیت‌های ۱۵ تا ۸ (۸ بیت): بخش حقیقی عملوند دوم.
- بیت‌های ۷ تا ۰ (۸ بیت): بخش موهومی عملوند دوم.

جدول زیر مجموعه دستوره‌های پیشنهادی در این معماری را نشان می‌دهد:

کد دستور	نام دستور	شرح عملیات
000	CADD	جمع دو عدد مختلط
001	CSUB	تفریق دو عدد مختلط
010	CMUL	ضرب دو عدد مختلط
011	CONJ	محاسبه مزدوج مختلط یک عدد
100	CMAG2	محاسبه اندازه مربع عدد مختلط
101	NOP	بدون عملیات

جدول ۲: نقشه کدهای عملیاتی

دستورهای مزدوج مختلط و اندازه مربع فقط به یک عملوند مختلط نیاز دارند. در این قالب طراحی، عملوند دوم در این دستورها نادیده گرفته شده و تاثیری در خروجی ندارد، لذا با مقدار صفر پر می‌شود.

۸ مدیریت پرچم‌های وضعیت

در پایان اجرای هر دستور، پرچم‌های وضعیت متناسب با خروجی‌های تولید شده در مسیر داده به‌روزرسانی می‌شوند. این پرچم‌ها به صورت ترکیبی پیاده شده‌اند اما بر اساس قرارداد طراحی، مقادیر آن‌ها تنها در سیکلی که سیگنال معتبر یک می‌شود (پایان دستور) معتبر و قابل استناد است. نحوه مدیریت این پرچم‌ها به شرح زیر است:

- **پرچم صفر:** این پرچم نشان می‌دهد نتیجه نهایی برابر صفر است. برای خروجی مختلط، با استفاده از یک درگاه مقایسه‌گر بررسی می‌شود که آیا هر دو بخش حقیقی و موهومی خروجی دقیقاً برابر با صفر هستند یا خیر.

¹²Opcode

- پرچم منفی بودن بخش حقیقی و موهومی: نشان می‌دهد بخش حقیقی یا موهومی خروجی منفی است. از آنجا که اعداد در نمایش مکمل دو هستند، پرارزش‌ترین بیت هر بخش مستقیماً این وضعیت را نشان می‌دهد.
- پرچم سرریز: نشان می‌دهد در نتیجه عملیات جمع، تفریق یا ضرب سرریز رخ داده است. برای مدیریت این وضعیت، یک تسهیم‌کننده هوشمند با بررسی کد دستور فعلی تصمیم می‌گیرد که آیا خطای سرریز را از جمع‌کننده اصلی، از پیمانه مجازی مزدوج و یا از پیمانه مجازی مجذور اندازه به خروجی نهایی منتقل کند.
- پرچم معتبر: نشان می‌دهد خروجی عملیات معتبر است و عملیات جاری به پایان رسیده است. این سیگنال در پایان آخرین چرخه اجرای هر دستور، توسط ماشین حالت منتهای فعال می‌گردد.

۹ پیمانه واحد کنترل

واحد کنترل با دریافت کد دستور، عملیات چند چرخه‌ای^{۱۳} را با استفاده از یک ماشین حالت منتهای^{۱۴} مدیریت و کنترل می‌کند.

۱.۹ پیاده‌سازی کد

```

1 module ControlUnit (
2     input clk,
3     input rst,
4     input start,
5     input [2:0] opcode,
6
7     output reg [2:0] sel_mul_1,
8     output reg [2:0] sel_mul_2,
9     output reg [2:0] sel_add_1,
10    output reg [2:0] sel_add_2,
11    output reg add_sub_ctrl, // 0 = add, 1 = sub
12    output reg en_reg1, en_reg2, en_reg3, en_reg4,
13    output reg en_addout_real, en_addout_imag, en_mulout_real, en_mulout_imag,
14    output reg valid
15 );
16
17 localparam IDLE      = 5'd0;
18 localparam CADD1     = 5'd1; localparam CADD2     = 5'd2;
19 localparam CSUB1     = 5'd3; localparam CSUB2     = 5'd4;
20 localparam CONJ1     = 5'd5; localparam CONJ2     = 5'd6;
21 localparam CMUL1     = 5'd7; localparam CMUL2     = 5'd8;
22 localparam CMUL3     = 5'd9; localparam CMUL4     = 5'd10;
23 localparam CMUL5     = 5'd11; localparam CMUL6     = 5'd12;
24 localparam CMUL7     = 5'd13;
25 localparam CMAG2_1   = 5'd14; localparam CMAG2_2   = 5'd15;
26 localparam CMAG2_3   = 5'd16; localparam CMAG2_4   = 5'd17;
27
28 reg [4:0] state;
29 reg [4:0] next_state;
30
31 always @(*) begin
32     next_state = state;
33     case(state)
34         IDLE: begin
35             if(start) begin
36                 case(opcode)
37                     3'b000: next_state = CADD1;
38                     3'b001: next_state = CSUB1;
39                     3'b010: next_state = CMUL1;

```

¹³Multi-Cycle Execution

¹⁴Finite State Machine

```

F0          3'b011: next_state = CONJ1;
F1          3'b100: next_state = CMAG2_1;
F2          default: next_state = IDLE;
F3      endcase
F4      end else begin
F5          next_state = IDLE;
F6      end
F7  end
F8      CADD1: next_state = CADD2; CADD2: next_state = IDLE;
F9      CSUB1: next_state = CSUB2; CSUB2: next_state = IDLE;
50      CONJ1: next_state = CONJ2; CONJ2: next_state = IDLE;
51      CMUL1: next_state = CMUL2; CMUL2: next_state = CMUL3;
52      CMUL3: next_state = CMUL4; CMUL4: next_state = CMUL5;
53      CMUL5: next_state = CMUL6; CMUL6: next_state = CMUL7;
54      CMUL7: next_state = IDLE;
55      CMAG2_1: next_state = CMAG2_2; CMAG2_2: next_state = CMAG2_3;
56      CMAG2_3: next_state = CMAG2_4; CMAG2_4: next_state = IDLE;
57      default: next_state = IDLE;
58  endcase
59  end
60
61  always @(posedge clk or posedge rst) begin
62      if(rst) state <= IDLE;
63      else state <= next_state;
64  end
65
66  always @(*) begin
67      sel_mul_1 = 3'b000; sel_mul_2 = 3'b000;
68      sel_add_1 = 3'b000; sel_add_2 = 3'b000;
69      add_sub_ctrl = 1'b0;
70      en_reg1 = 1'b0; en_reg2 = 1'b0; en_reg3 = 1'b0; en_reg4 = 1'b0;
71      en_addout_real = 1'b0; en_addout_imag = 1'b0;
72      en_mulout_real = 1'b0; en_mulout_imag = 1'b0;
73      valid = 1'b0;
74
75      case(state)
76          CADD1: begin sel_add_1=3'b000; sel_add_2=3'b010; add_sub_ctrl=1'b0;
77                  en_addout_real=1'b1; end
78          CADD2: begin sel_add_1=3'b001; sel_add_2=3'b011; add_sub_ctrl=1'b0;
79                  en_addout_imag=1'b1; valid=1'b1; end
80          CSUB1: begin sel_add_1=3'b000; sel_add_2=3'b010; add_sub_ctrl=1'b1;
81                  en_addout_real=1'b1; end
82          CSUB2: begin sel_add_1=3'b001; sel_add_2=3'b011; add_sub_ctrl=1'b1;
83                  en_addout_imag=1'b1; valid=1'b1; end
84          CONJ1: begin sel_add_1=3'b000; sel_add_2=3'b010; add_sub_ctrl=1'b0;
85                  en_addout_real=1'b1; end
86          CONJ2: begin sel_mul_1=3'b001; sel_mul_2=3'b100; en_mulout_imag=1'b1; valid
87                  =1'b1; end
88          CMUL1: begin sel_mul_1=3'b000; sel_mul_2=3'b010; en_reg1=1'b1; end
89          CMUL2: begin sel_mul_1=3'b001; sel_mul_2=3'b011; en_reg2=1'b1; end
90          CMUL3: begin sel_mul_1=3'b000; sel_mul_2=3'b011; en_reg3=1'b1; end
91          CMUL4: begin sel_mul_1=3'b001; sel_mul_2=3'b010; en_reg4=1'b1; end
92          CMUL5: begin sel_add_1=3'b100; sel_add_2=3'b101; add_sub_ctrl=1'b1;
93                  en_addout_real=1'b1; end
94          CMUL6: begin sel_add_1=3'b110; sel_add_2=3'b111; add_sub_ctrl=1'b0;
95                  en_addout_imag=1'b1; end
96          CMUL7: begin valid=1'b1; end
97          CMAG2_1: begin sel_mul_1=3'b000; sel_mul_2=3'b000; en_reg1=1'b1; end
98          CMAG2_2: begin sel_mul_1=3'b001; sel_mul_2=3'b001; en_reg2=1'b1; end
99          CMAG2_3: begin sel_add_1=3'b100; sel_add_2=3'b101; add_sub_ctrl=1'b0;
100                  en_addout_real=1'b1; end
101          CMAG2_4: begin sel_add_1=3'b100; sel_add_2=3'b100; add_sub_ctrl=1'b1;
102                  en_addout_imag=1'b1; valid=1'b1; end

```

```

۹۳     endcase
۹۴ end
۹۵ endmodule

```

کد پیمانه واحد کنترل

توضیح خط به خط کد:

- خطوط ۱۷ تا ۲۷: تخصیص پارامترهای ثابت برای کدگذاری حالات ماشین وضعیت. این روش خوانایی کد را به شدت بالا می‌برد.
- خطوط ۳۲ تا ۵۸: بلوک ترکیبی برای تعیین حالت بعدی^{۱۵}. این بخش با بررسی کد دستور، تصمیم می‌گیرد که ماشین از حالت استراحت به کدام زیرشاخه هدایت شود و ترتیب پرش‌ها را تا اتمام دستور مشخص می‌کند.
- خطوط ۶۰ تا ۶۳: بلوک منطق ترتیبی (همگام با ساعت) که وضعیت فعلی سیستم را به روزرسانی می‌کند و یا آن را بازنشانی می‌نماید.
- خطوط ۶۵ تا ۹۶: بلوک تولید خروجی‌های ترکیبی^{۱۶}. در این بخش بسته به حالت فعلی ماشین، سیگنال‌های کنترلی مسیر داده (نظیر فعال‌سازهای ثبات‌ها و مسیرهای تسهیم‌کننده) یک می‌شوند و در آخرین فاز، سیگنال پایان عملیات نیز فعال می‌گردد.

۲.۹ زمان‌بندی و مراحل اجرای دستورات چندچرخه‌ای

اجرای عملیات پیچیده مستلزم اشتراک‌گذاری منابع سخت‌افزاری^{۱۷} است. جداول زیر سیگنال‌های کنترلی مسیر داده و عملیات ریاضی در هر چرخه را کالبدشکافی می‌کنند.

۱.۲.۹ دستور جمع دو عدد مختلط

چرخه	عملیات ریاضی	عملیات در مسیر داده	سیگنال‌های فعال
۱	$a + c$	محاسبه بخش حقیقی	sel_add_1=000, sel_add_2=010, en_addout_real=1
۲	$b + d$	محاسبه بخش موهومی	sel_add_1=001, sel_add_2=011, en_addout_imag=1, valid=1

۲.۲.۹ دستور تفریق دو عدد مختلط

چرخه	عملیات ریاضی	عملیات در مسیر داده	سیگنال‌های فعال
۱	$a - c$	محاسبه بخش حقیقی	sel_add_1=000, sel_add_2=010, add_sub_ctrl=1, en_addout_real=1
۲	$b - d$	محاسبه بخش موهومی	sel_add_1=001, sel_add_2=011, add_sub_ctrl=1, en_addout_imag=1, valid=1

۳.۲.۹ دستور محاسبه مزدوج مختلط یک عدد

چرخه	عملیات ریاضی	عملیات در مسیر داده	سیگنال‌های فعال
۱	a	عبور بخش حقیقی به خروجی	en_addout_real=1
۲	$b \times (-1)$	قرینه‌سازی بخش موهومی	sel_mul_1=001, sel_mul_2=100, en_mulout_imag=1, valid=1

¹⁵Next State Logic

¹⁶Output Logic

¹⁷Resource Sharing

۴.۲.۹ دستور ضرب دو عدد مختلط

چرخه	عملیات ریاضی	عملیات در مسیر داده	سیگنال‌های فعال
۱	ac	ضرب و ذخیره در ثبات ۱	<code>sel_mul_1=000, sel_mul_2=010, en_reg1=1</code>
۲	bd	ضرب و ذخیره در ثبات ۲	<code>sel_mul_1=001, sel_mul_2=011, en_reg2=1</code>
۳	ad	ضرب و ذخیره در ثبات ۳	<code>sel_mul_1=000, sel_mul_2=011, en_reg3=1</code>
۴	bc	ضرب و ذخیره در ثبات ۴	<code>sel_mul_1=001, sel_mul_2=010, en_reg4=1</code>
۵	$ac - bd$	تفریق برای بخش حقیقی خروجی	<code>sel_add_1=100, sel_add_2=101, add_sub_ctrl=1, en_addout_real=1</code>
۶	$ad + bc$	جمع برای بخش موهومی خروجی	<code>sel_add_1=110, sel_add_2=111, add_sub_ctrl=0, en_addout_imag=1</code>
۷	-	تولید خروجی نهایی	<code>valid=1</code>

۵.۲.۹ دستور محاسبه اندازه مربع عدد مختلط

چرخه	عملیات ریاضی	عملیات در مسیر داده	سیگنال‌های فعال
۱	a^2	محاسبه و ذخیره در ثبات ۱	<code>sel_mul_1=000, sel_mul_2=000, en_reg1=1</code>
۲	b^2	محاسبه و ذخیره در ثبات ۲	<code>sel_mul_1=001, sel_mul_2=001, en_reg2=1</code>
۳	$a^2 + b^2$	محاسبه مجموع برای بخش حقیقی	<code>sel_add_1=100, sel_add_2=101, add_sub_ctrl=0, en_addout_real=1</code>
۴	۰	صفر کردن بخش موهومی و خروجی نهایی	<code>sel_add_1=100, sel_add_2=100, add_sub_ctrl=1, en_addout_imag=1, valid=1</code>

۱۰ آزمون نهایی سیستم

یک واحد خط لوله^{۱۸} طراحی شده است که وظیفه اتصال اجزا و هماهنگی خواندن از یک حافظه ۳۲ کلمه‌ای را بر عهده دارد. طراحی کاملاً هم‌زمان^{۱۹} و مبتنی بر ساعت^{۲۰} است.

۱.۱۰ پیاده‌سازی کد میزآزمون نهایی

برای طراحی نهایی یک میزآزمون جامع نوشته شده است که چند دستور متوالی را در حافظه قرار داده و اجرای صحیح آن‌ها را بررسی می‌کند. این میزآزمون برای اطمینان از مقیاس‌پذیری و صحت عملکرد ماژول نهایی، سه نسخه مجزا از ماشین را با عرض‌های ۴، ۸ و ۱۶ بیتی شبیه‌سازی می‌کند.

```

۱ `timescale 1ns / 1ps
۲
۳ module tb;
۴     parameter CLK_PERIOD = 10;
۵     reg clk;
۶     reg rst;
۷
۸     // 8-bit System Instance
۹     wire signed [15:0] fr8, fi8;
۱0    wire fz8, fo8, fnr8, fni8, done8;
۱۱    Top #(8) uut8 (.clk(clk), .rst(rst), .final_out_real(fr8), .final_out_imag(fi8),
۱۲        .final_zero(fz8), .final_overflow(fo8), .final_neg_real(fnr8), .
        final_neg_imag(fni8), .done_all(done8));
۱۳
۱۴    // 4-bit System Instance
۱۵    wire signed [7:0] fr4, fi4;
۱۶    wire fz4, fo4, fnr4, fni4, done4;
۱۷    Top #(4) uut4 (.clk(clk), .rst(rst), .final_out_real(fr4), .final_out_imag(fi4),
۱۸        .final_zero(fz4), .final_overflow(fo4), .final_neg_real(fnr4), .
        final_neg_imag(fni4), .done_all(done4));
۱۹
۲۰    // 16-bit System Instance
۲۱    wire signed [31:0] fr16, fi16;
۲۲    wire fz16, fo16, fnr16, fni16, done16;

```

¹⁸Pipeline

¹⁹Synchronous

²⁰Clock

```

۲۳ Top #(16) uut16 (.clk(clk), .rst(rst), .final_out_real(fr16), .final_out_imag(
۲۴     fi16),
۲۵     .final_zero(fz16), .final_overflow(fo16), .final_neg_real(fnr16
۲۶     ), .final_neg_imag(fni16), .done_all(done16));
۲۷
۲۸ initial begin
۲۹     clk = 0;
۳۰     forever #(CLK_PERIOD/2) clk = ~clk;
۳۱ end
۳۲
۳۳ initial begin
۳۴     // Allow $readmemh to initialize uut8 naturally
۳۵     #1;
۳۶
۳۷     // Manual override for 4-bit memory (Add: 2+3j + 1+1j)
۳۸     uut4.memory[0] = {3'b000, 4'sd2, 4'sd3, 4'sd1, 4'sd1};
۳۹     uut4.memory[1] = {3'b101, 16'd0}; // NOP
۴۰
۴۱     // Manual override for 16-bit memory (Add: 1000+2000j + 500+100j)
۴۲     uut16.memory[0] = {3'b000, 16'sd1000, 16'sd2000, 16'sd500, 16'sd100};
۴۳     uut16.memory[1] = {3'b101, 64'd0}; // NOP
۴۴
۴۵     // Initiate computation
۴۶     rst = 1;
۴۷     #(CLK_PERIOD * 2);
۴۸     rst = 0;
۴۹
۵۰     // Wait for all scaling tests to finish
۵۱     wait (done8 == 1'b1 && done4 == 1'b1 && done16 == 1'b1);
۵۲
۵۳     #(CLK_PERIOD * 5);
۵۴     $display("=====");
۵۵     $display("All test cases across 4-bit, 8-bit, and 16-bit executed
۵۶     successfully!");
۵۷     $display("=====");
۵۸     $stop;
۵۹ end
۶۰ endmodule

```

کد میزآزمون نهایی سیستم

توضیح خط به خط کد:

- خطوط ۸ تا ۲۴: نمونه‌سازی هم‌زمان ۳ ماشین کامپیوتر کاملاً مجزا با عرض‌های ۸، ۴ و ۱۶ بیت جهت انجام تست جامع مقیاس‌پذیری سخت‌افزار.
- خط ۳۵ تا ۳۹: بازنویسی دستی^{۲۱} فضای حافظه در ماژول‌های ۴ و ۱۶ بیتی جهت تزریق دستورات آزمایشی، بدون ایجاد اختلال در خواندن فایل اصلی حافظه برای سیستم ۸ بیتی.
- خطوط ۴۹ تا ۵۴: ایجاد شرط توقف هوشمند با استفاده از عملگر wait برای اطمینان از اتمام موفقیت‌آمیز محاسبات در هر ۳ ماشین پیش از چاپ پیام نهایی در کنسول و متوقف ساختن شبیه‌سازی.

۲.۱۰ تحلیل موارد آزمون (محتوای حافظه)

۹ دستور مختلف در حافظه نسخه پایه (۸ بیتی) از طریق فایل متن به فرمت هگزادسیمال مقداردهی شده‌اند. در ادامه روند اجرای این دستورات تحلیل شده است:

²¹Override

۱.۲.۱۰ دستور اول: جمع دو عدد مختلط مثبت

ساختار دستور ۳۵ بیتی: 000_00000010_00000011_00000100_00000001

• کد عملیات: 000 (جمع)

• عملوند اول: $A = 2 + 3j$

• عملوند دوم: $B = 4 + 1j$

شرح عملیات: $z = 6 + 4j = (4 + 1j) + (2 + 3j)$. در این حالت، انتظار می‌رود که پس از ۲ چرخه، خروجی برابر با مجموع دو عدد شود و هیچ پرچم خطایی روشن نگردد.

۲.۲.۱۰ دستور دوم: جمع دو عدد مختلط منفی

ساختار دستور ۳۵ بیتی: 000_11111011_11111110_11111111_11111101

• کد عملیات: 000 (جمع)

• عملوند اول: $A = -5 - 2j$ (مقادیر FB و FE در هگزادسیمال)

• عملوند دوم: $B = -1 - 3j$ (مقادیر FF و FD در هگزادسیمال)

شرح عملیات: $z = -6 - 5j = (-1 - 3j) + (-5 - 2j)$. پس از ۲ چرخه، پاسخ صحیح در خروجی ظاهر می‌شود و پرچم‌های مربوط به منفی بودن فعال می‌گردند.

۳.۲.۱۰ دستور سوم: تفریق دو عدد مختلط با نتیجه منفی

ساختار دستور ۳۵ بیتی: 001_00000011_00000010_00001000_00000101

• کد عملیات: 001 (تفریق)

• عملوند اول: $A = 3 + 2j$

• عملوند دوم: $B = 8 + 5j$

شرح عملیات: $z = -5 - 3j = (3 + 2j) - (8 + 5j)$. مدار عملیات تفریق را انجام داده و پرچم‌های مربوط به اعداد منفی روشن می‌شوند.

۴.۲.۱۰ دستور چهارم: ضرب دو عدد مختلط با مؤلفه‌های مثبت و منفی

ساختار دستور ۳۵ بیتی: 010_00000010_11111101_11111111_00000100

• کد عملیات: 010 (ضرب)

• عملوند اول: $A = 2 - 3j$ (مقدار FD برای بخش موهومی)

• عملوند دوم: $B = -1 + 4j$ (مقدار FF برای بخش حقیقی)

شرح عملیات: $z = 10 + 11j = (2 - 3j) \times (-1 + 4j)$. پس از طی شدن ۷ چرخه کامل محاسباتی، خروجی به درستی تولید می‌شود که صحت عملکرد شیفت‌ها و محاسبات متوالی در ماشین حالت را ثابت می‌کند.

۵.۲.۱۰ دستور پنجم: محاسبه مزدوج مختلط

ساختار دستور ۳۵ بیتی: 011_00000100_00000111_00000000_00000000

• کد عملیات: 011 (مزدوج)

• عملوند اول: $A = 4 + 7j$

• عملوند دوم: (در این دستور با صفر پر شده است)

شرح عملیات: $z = 4 - 7j$. $(4 + 7j)^* = 4 - 7j$. مدار بخش حقیقی را حفظ کرده و بخش موهومی را در منفی یک ضرب می‌کند تا حاصل تولید شود.

۶.۲.۱۰ دستور ششم: محاسبه اندازه مربع عدد مختلط

ساختار دستور ۳۵ بیتی: 100_00000011_11111100_00000000_00000000

• کد عملیات: 100 (اندازه مربع)

• عملوند اول: $A = 3 - 4j$

• عملوند دوم: (در این دستور با صفر پر شده است)

شرح عملیات: $25 = 9 + 16 = (-4)^2 + 3^2 = |3 - 4j|^2$. مدار پس از ۴ چرخه، عدد نهایی ۲۵ را برای بخش حقیقی و صفر را برای بخش موهومی گزارش می‌دهد.

۷.۲.۱۰ دستور هفتم: بررسی خروجی صفر و فعال شدن پرچم مربوطه

ساختار دستور ۳۵ بیتی: 001_00000101_00000101_00000101_00000101

• کد عملیات: 001 (تفریق)

• عملوند اول: $A = 5 + 5j$

• عملوند دوم: $B = 5 + 5j$

شرح عملیات: $z = 0 + 0j$. $(5 + 5j) - (5 + 5j) = 0$. تفریق دو عدد یکسان منجر به خروجی مطلق صفر شده و پرچم مربوطه در مدار فعال می‌شود.

۸.۲.۱۰ دستور هشتم: بررسی حالت‌های مرزی برای وقوع سرریز

ساختار دستور ۳۵ بیتی: 000_01100100_01100100_01100100_01100100

• کد عملیات: 000 (جمع)

• عملوند اول: $A = 100 + 100j$ (مقدار 64 در هگزادسیمال)

• عملوند دوم: $B = 100 + 100j$

شرح عملیات: $z = 200 + 200j$. $(100 + 100j) + (100 + 100j) = 200 + 200j$. جمع این بردارها از بیشترین مقدار مثبت قابل نمایش در سیستم هشت بیتی (۱۲۷) عبور می‌کند. این عملیات باعث تغییر علامت نامعتبر شده و پرچم مربوطه یک می‌گردد.

۹.۲.۱۰ دستور نهم: بدون عملیات

ساختار دستور ۳۵ بیتی: 101_00000000_00000000_00000000_00000000

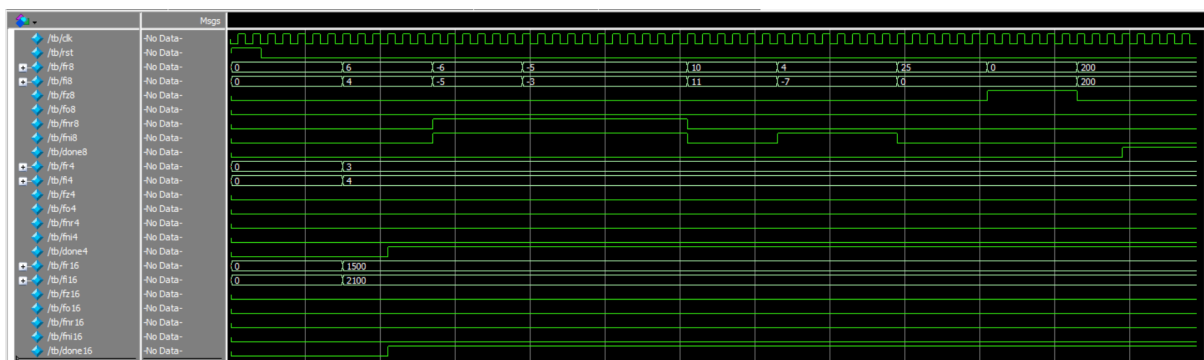
• کد عملیات: 101 (بدون عملیات)

شرح عملیات: این دستور به واحد خط لوله اعلام می‌کند که پایان محتوای مفید حافظه فرا رسیده است.

۱۰.۲.۱۰ آزمون مقیاس پذیری (۴ و ۱۶ بیتی)

جهت اطمینان از مقیاس پذیری، دو دستور به طور دستی در حافظه سیستم های ۴ و ۱۶ بیتی جای گذاری گردید:

- تست ۴ بیتی: جمع مقادیر $z3 + z2 + z1 + 1$. خروجی مورد انتظار $z3 + 4$ است که در خروجی های $fr4$ و $fi4$ مشاهده می شود.
- تست ۱۶ بیتی: جمع مقادیر $z2000 + 1000 + z1000 + 500$. خروجی مورد انتظار $z2100 + 1500$ است که در خروجی های $fr16$ و $fi16$ قرار می گیرد.



شکل ۵: شکل موج خروجی یکپارچه مدار پس از اجرای تمامی دستورات

۱۱ بررسی قابلیت سنتز در نرم افزار Quartus

یک پروژه در نرم افزار Quartus ساخته شد و فایل های طراحی به آن افزوده شد. نام پیمانه سطح بالا^{۲۲} انتخاب شد. برای بررسی اولیه قابل سنتز بودن طراحی، مرحله تحلیل و سنتز از مسیر:

Processing > Start > Start Analysis & Synthesis

اجرا شد. سپس برای تولید نتایج مرحله تطبیق و امکان مشاهده نمای پس از جایابی، کامپایل کامل از مسیر:

Processing > Start Compilation

انجام گرفت.

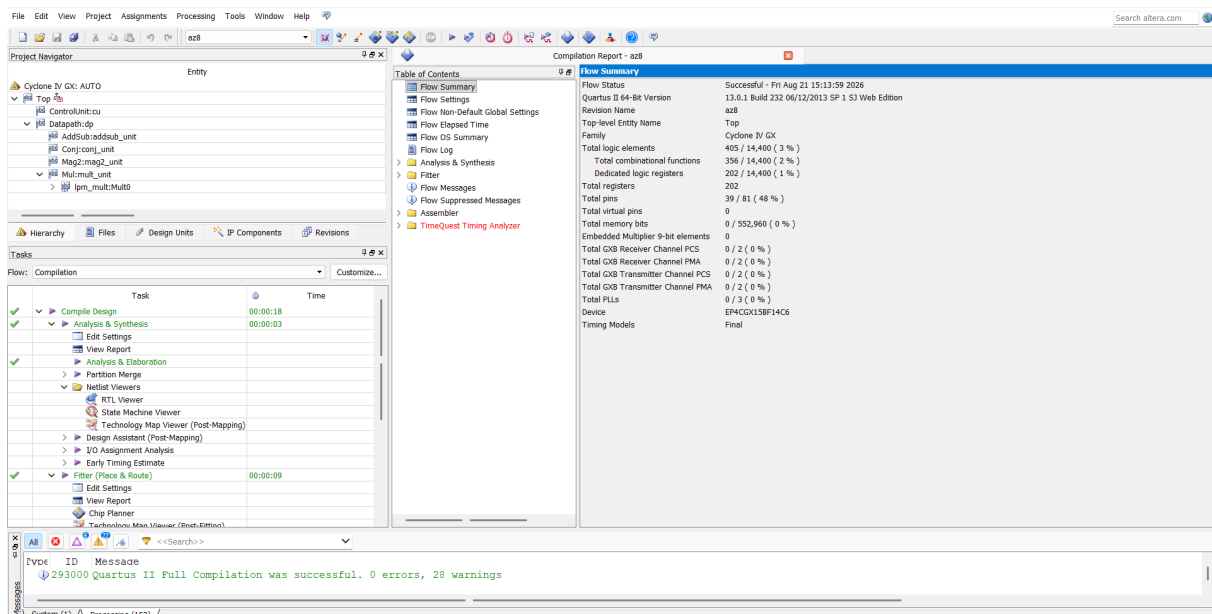
پس از تکمیل کامپایل، سه نمای اصلی از مسیرهای زیر باز شدند:

Tools > Netlist Viewers > RTL Viewer

Tools > Netlist Viewers > Technology Map Viewer (Post-Mapping)

Tools > Netlist Viewers > Technology Map Viewer (Post-Fitting)

در ادامه، خروجی موفقیت آمیز بودن کامپایل و همچنین نماهای بررسی شده به ترتیب آورده شده است. نمای سطح انتقال ثبات در واقع یک فهرست اتصالات^{۲۳} بصری است و نرم افزار Quartus با تشکیل سلسله مراتب^{۲۴} مدار، پروسه را بدون خطا طی کرده است.

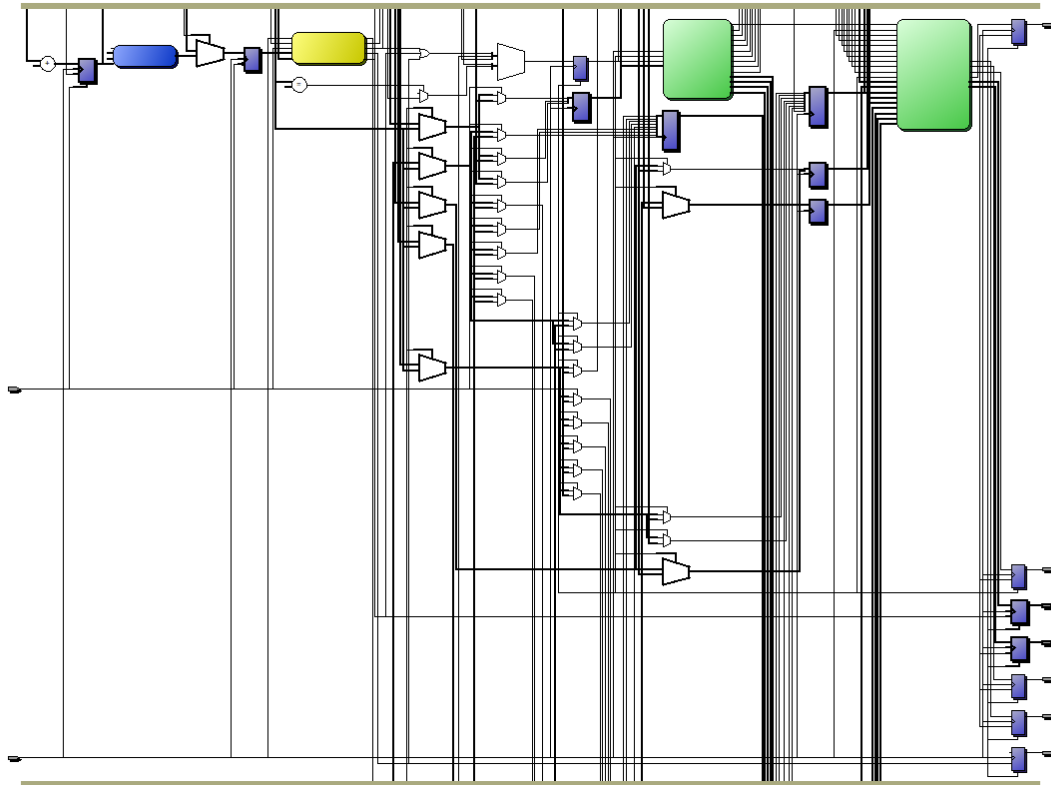


شکل ۶: گزارش موفقیت آمیز بودن کامپایل کامل و فرآیند سنتز در Quartus

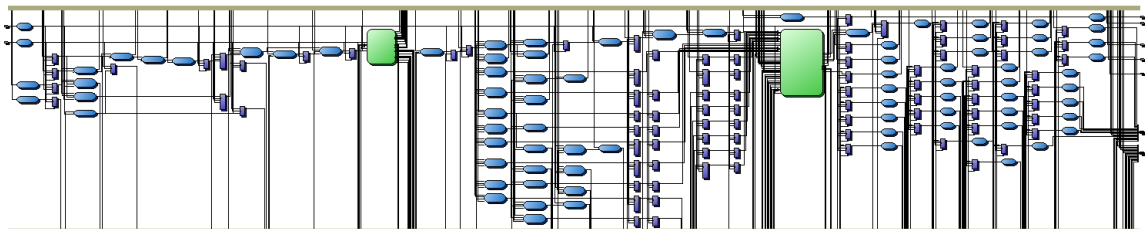
²²Top-Level Entity

²³Netlist

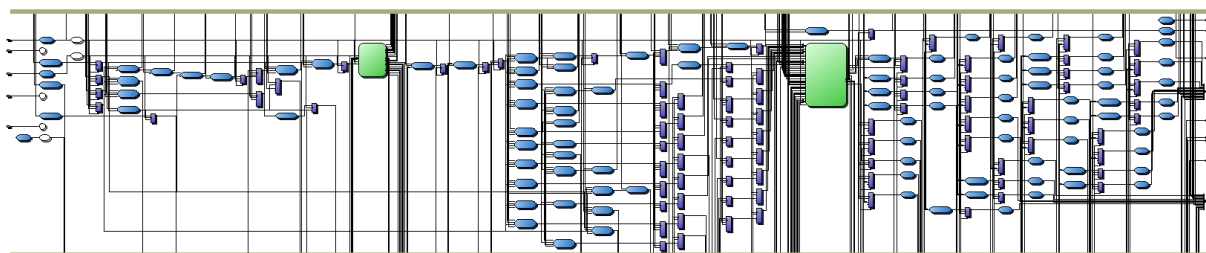
²⁴Hierarchy



شکل ۷: نمای سطح انتقال ثبات (RTL Viewer)



شکل ۸: نمای نگاشت فناوری پس از نگاشت (Post-Mapping)



شکل ۹: نمای نگاشت فناوری پس از جاییابی و تطبیق (Post-Fitting)